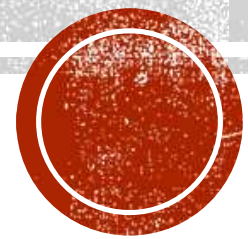
STATE SPACE SEARCH TECHNIQUES IN AI WITH SOME APPLICATIONS



CONTENT

- Breadth first search, Depth first search
- Best first search
- A* algorithm
- AO* Algorithm
- Hill climbing
- Minimax
- Constraint Satisfaction

- **Failure is the opportunity to begin again more intelligently.**

-Moshe Arens (1925, Israeli politician)

SEARCHING STRATEGIES

•**Blind search (Uninformed)** → traversing the search space until the goal nodes is found (might be doing exhaustive search).

•*Techniques* : **Breadth First, ,Depth first…**

•Guarantees solution.

•**Heuristic search (Informed)** → search process takes place by traversing search space with applied rules (information).

•T*echniques*: **Greedy Best First Search, A* Algorithm, Hill Climbing…**

•There is no guarantee that solution is found.

UNINFORMED VS INFORMED SEARCH

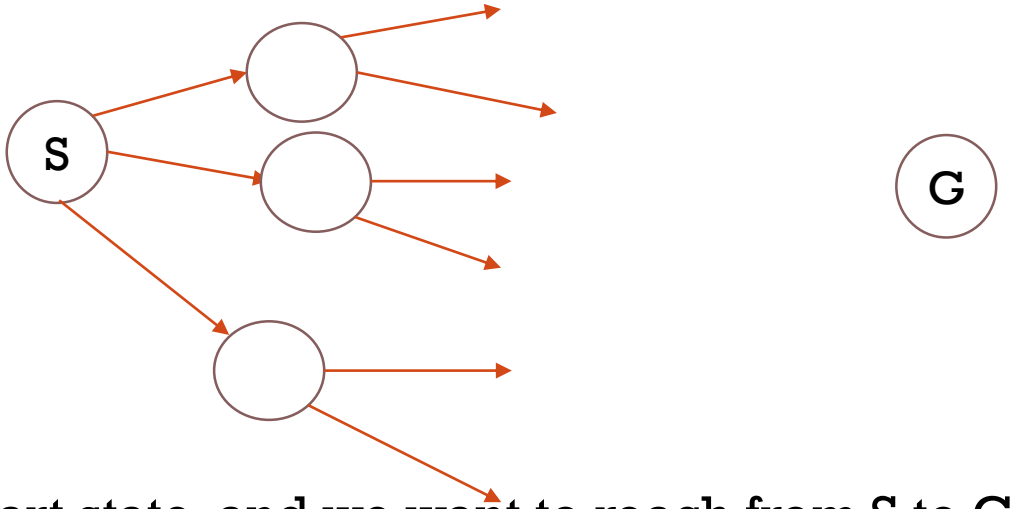
Uninformed Search	Informed Search
Search without information	Search with information
No knowledge	Use knowledge to find steps to the solution
Time consuming	Quick solution
More complexity (time, space)	Less complexity (time, space)
DFS, BFS etc…	A*, Best first search etc….

BLIND VS HEURISTIC

- A blind search is usually uninformed. It doesn't have any specific knowledge about the problem,
  - whereas a heuristic search is that which has information about the problem and therefore uses logic in decision making.
- What is Heuristics?
- **A heuristic is a technique that is used to solve a problem faster than the classic methods.**
- These techniques are used to find the approximate solution of a problem when classical methods do not.
- Heuristics are said to be the problem-solving techniques that result in practical and quick solutions.
- Heuristics are **strategies that are derived from past experience** with similar problems. Heuristics use practical methods and shortcuts used to produce the solutions that may or may not be optimal, but those solutions are sufficient in a given limited timeframe.

UNINFORMED SEARCH

- **Uninformed:** searching without information
- **Informed:** searching with information
- Let us consider an example (Uninformed)



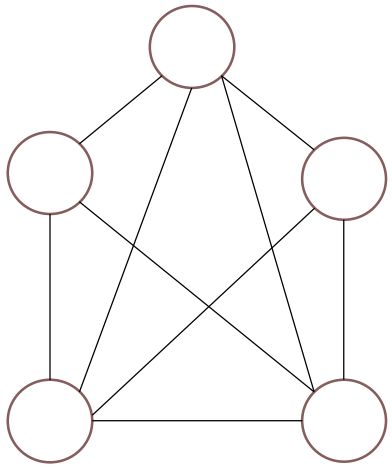
- S is the start state, and we want to reach from S to Goal state.

UNINFORMED SEARCH

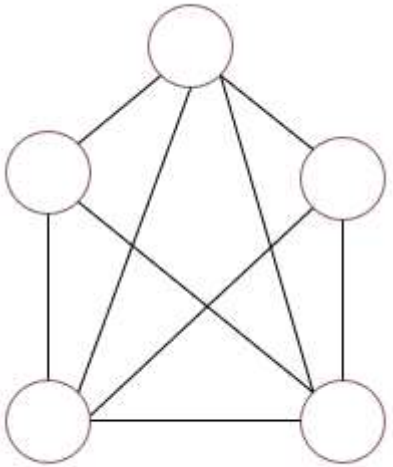
- Here, we explore all other states from the start state.
- Further we explore the other state at next level.
- Like this, after searching all the state space, finally we can reach to our goal state.
- Here, we have only knowledge of start sate and goal state, but there is no knowledge of domain or problem,
  - OR we can say there is no availability of guide who can help to select the particular path.

INFORMED SEARCH

- Here we use information, that is known as heuristic.
- Example: Consider TSP (Traveling salesman problem)



IF WE APPLY UNINFORMED SEARCH ON TSP (N CITIES)



- Check all the possibilities.
- So (n-1)! possibilities for TSP problem.
- So searching space would be (n-1)! .
- If we consider no. of cities 5, i.e. n=5, so (5-1) != 24 possibility/states, after covering all these states, finally you will be able to get optimal solution.
- So there is the exponential time growth, if n=100 OR some bigger numbers.
- Here no domain knowledge, only knowledge of start and goal states without any thumb rule.

IF WE APPLY INFORMED SEARCH ON TSP (N CITIES)

- In informed search, we use the heuristic method.
- How to compute the heuristic value?
- When to use heuristic?
  - When state space grows in exponential time, then time complexity is very high, that problem is known as NP problem. So to solve NP problems cost will be increased.
- If we are able to solve NP problems in polynomial time, then we can use heuristic methods.
- So in TSP (we use nearest neighbor), we find the heuristic value of each node, it will help to move further.
- Here we may compromise with optimality, heuristic method will give you good solution, but may OR may not give the optimal solution.

HEURISTIC IN AI: RULE OF THUMB (WHAT?, WHY?, HOW?)

- It is a technique designed to solve a problem quickly.
- Here kind of guess/ heuristic value.

- When to use state space search?
  - If we are able to represent the problem in set of states

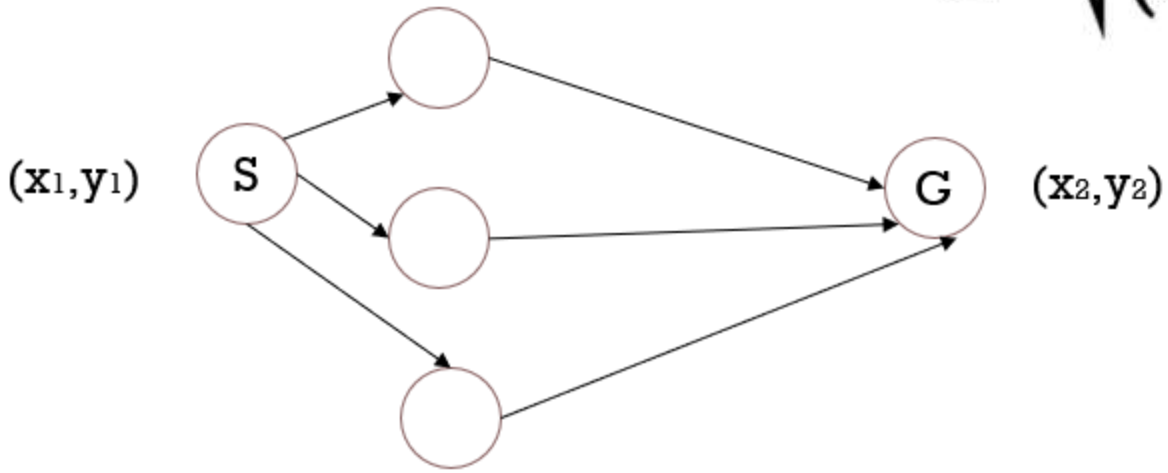
HOW TO CALCULATE HEURISTIC VALUE ?

heuristic value h(n)

Euclidean distance (straight-line distance)

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Example:



- To reach to the Goal state, we can calculate the distance between tow cities and take further decision to move with suitable path.
- But in case of blind search, we have to explore multiple path.

HOW TO CALCULATE HEURISTIC VALUE?

Manhattan Distance (to find out vertical/horizontal distance)

- Example: 8-puzzle problem

1	3	2
6	5	4
	8	7

Start state

1	2	3
4	5	6
7	8	

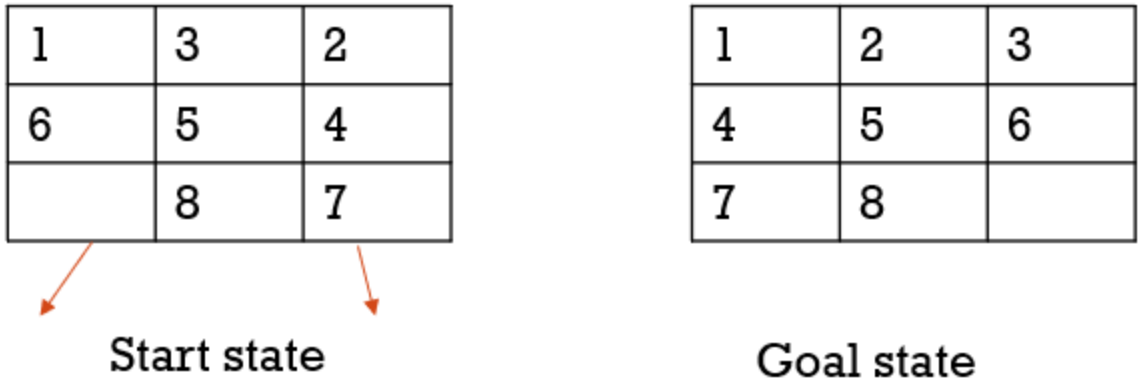
Goal state

- Manhattan Distance: d=0+1+1+2+0+2+2+0
- Any state at the next level of start state having less value of Manhattan Distance, we need to explore that path.

HOW TO CALCULATE HEURISTIC VALUE?

Another method, i.e. No of Misplaced Tiles:

- **Check which element is not at write position?**



- **No. of misplaced tiles=1+1+1+1+1=5 ?????**

MEASURING SEARCHING PERFORMANCE

- The output from problem-solving (searching) algorithm is either **FAILURE or SOLUTION**.
- Four ways:
  - **Completeness** : is guaranteed to find a solution?
  - **Optimality**: does it find optimal solution ?
  - **Time complexity**: how long?
  - **Space complexity**: how much memory?
    - Complexity : branching factor (*b*), depth (*d*), and max. depth (*m*)

EXAMPLE #1

- 8-puzzle problem without Heuristic in Artificial Intelligence
- 8-Puzzle Problem with Heuristic (Informed Search) in Artificial Intelligence

1	2	3
	4	6
7	5	8

Start state

1	2	3
4	5	6
7	8	

Goal state

EXAMPLE #2

2	8	3
1	6	4
7		5

Start state

1	2	3
8		4
7	6	5

Goal state

BEST FIRST SEARCH

Best First Search Algorithm

1. Create 2 empty lists: OPEN and CLOSED
2. Start from the initial node (say N) and put it in the 'ordered' OPEN list
3. Repeat the next steps until the GOAL node is reached
    1. If the OPEN list is empty, then EXIT the loop returning 'False'
    2. Select the first/top node (say N) in the OPEN list and move it to the CLOSED list. Also, capture the information of the parent node
    3. If N is a GOAL node, then move the node to the Closed list and exit the loop returning 'True'. The solution can be found by backtracking the path
    4. If N is not the GOAL node, expand node N to generate the 'immediate' next nodes linked to node N and add all those to the OPEN list
    5. Reorder the nodes in the OPEN list in ascending order according to an evaluation function f(n)

ILLUSTRATION OF BEST FIRST SEARCH

- In this assignment, we have to find out the shortest path between I (initial node) and F (final node).
- Now, if we would have to do that, we would need something called, heuristic values.
- What are these heuristic values? It is the straight line distances (Euclidean distances) from each node to the final node.

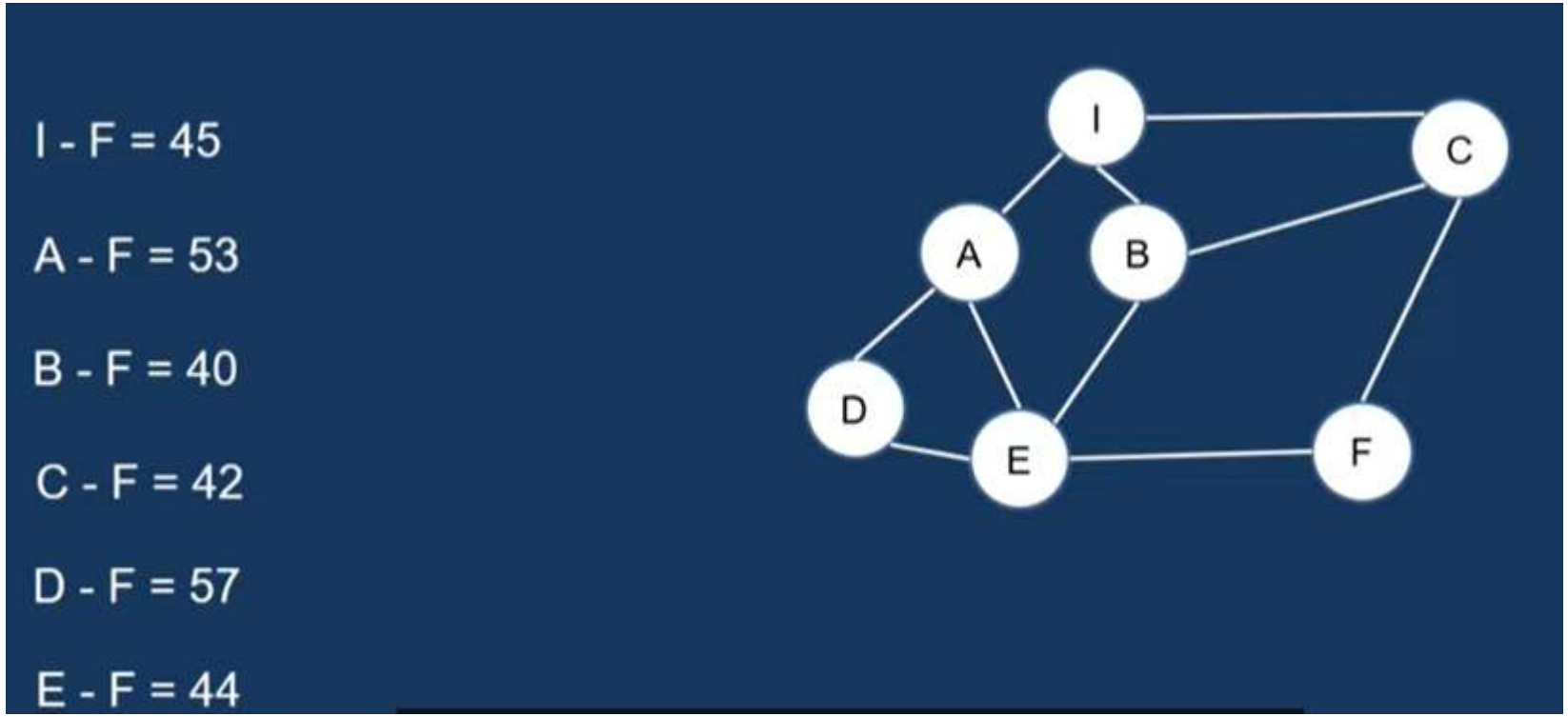
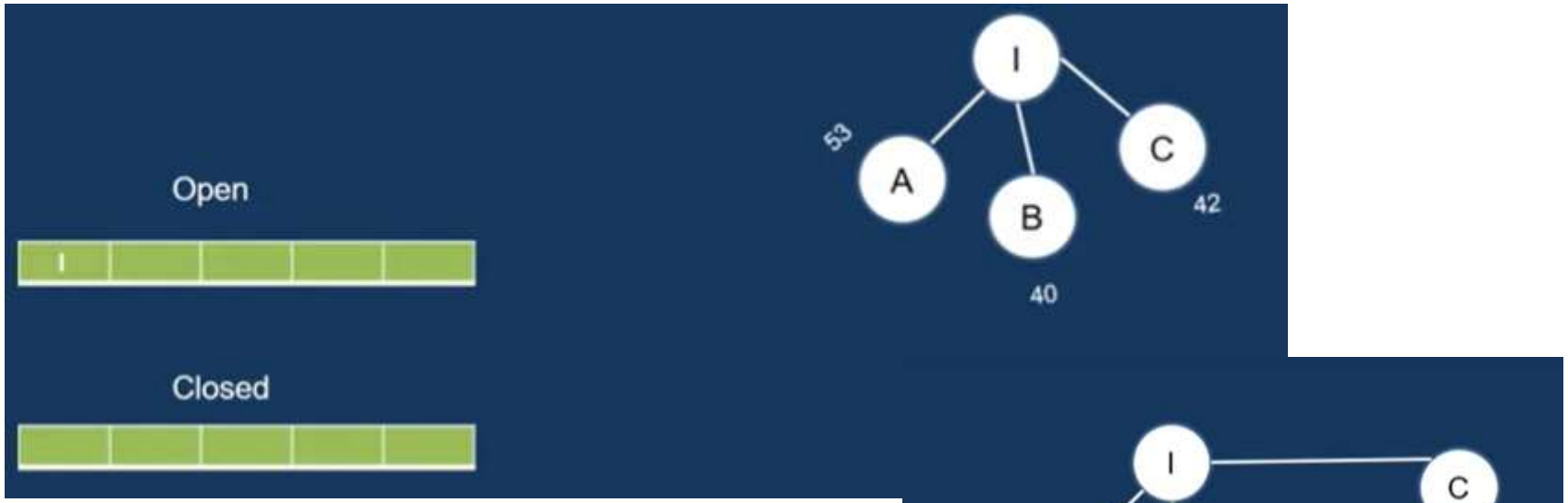


ILLUSTRATION OF BEST FIRST SEARCH

- Now, when it comes to best first search, you will have two priority queues, OPEN and CLOSED.
- Each node that we have to start traversing that we have to keep in OPEN, then list the successor "all the nodes which are directly connected".



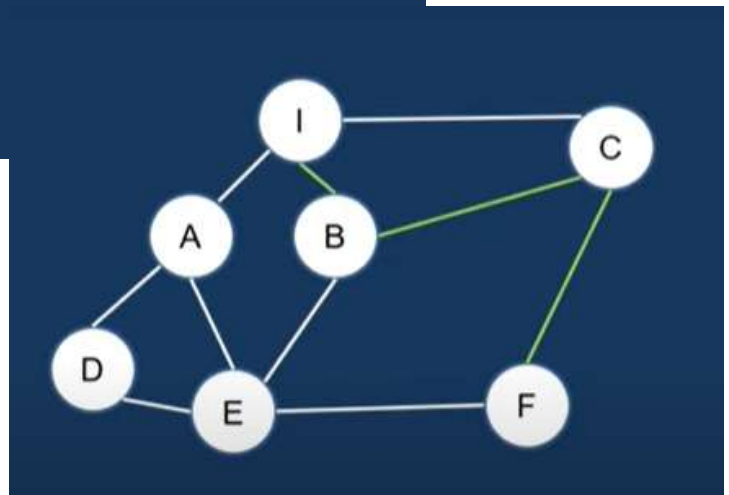


ILLUSTRATION OF BEST FIRST SEARCH

- Once we have successors listed down, we would have to take these and put them up in the OPEN priority queue.
- They will be stored in ascending order of their heuristic values.
- I has been already traversed, so it will be kept in CLOSED priority queue.



ILLUSTRATION OF BEST FIRST SEARCH

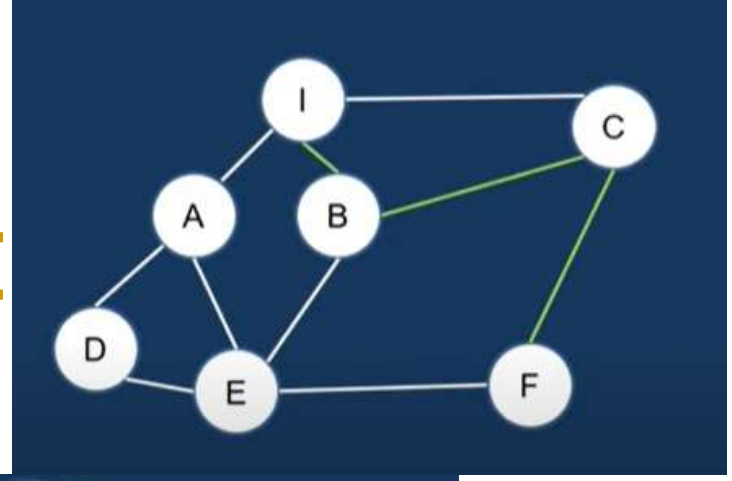
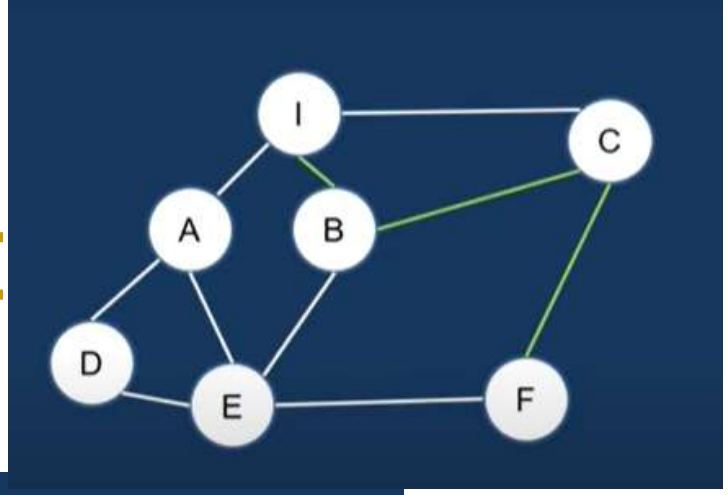




ILLUSTRATION OF BEST FIRST SEARCH



Open

C 42	E 44	A 53		

Closed

I	B			

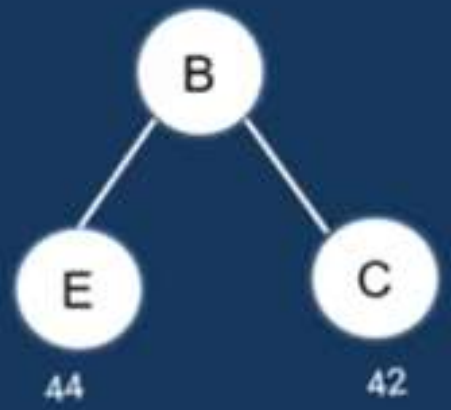
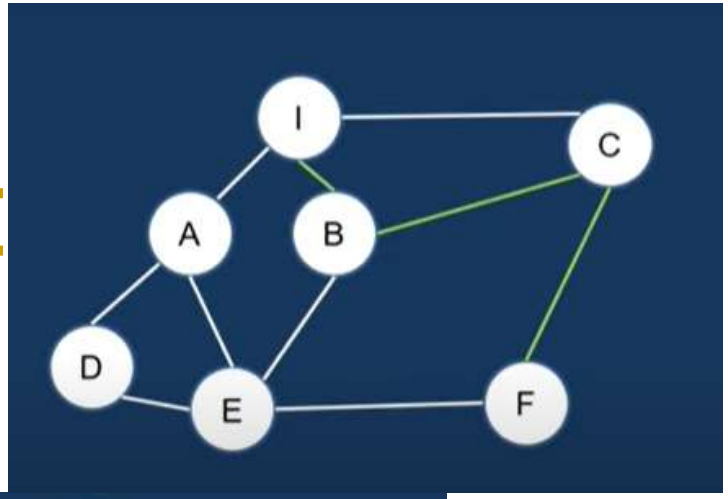


ILLUSTRATION OF BEST FIRST SEARCH



Open

C 42	E 44	A 53		

Closed

I	B			

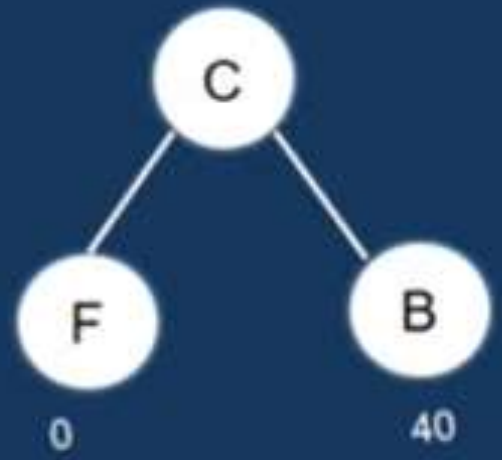
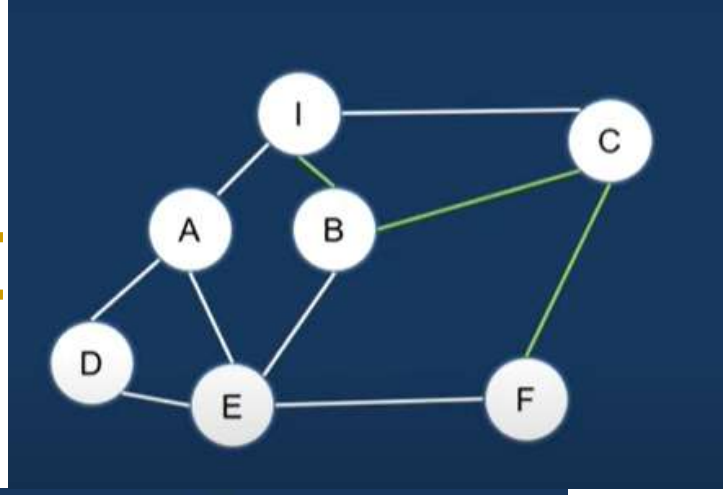


ILLUSTRATION OF BEST FIRST SEARCH



Open

F 0	E 44	A 53		

Closed

I	B	C		

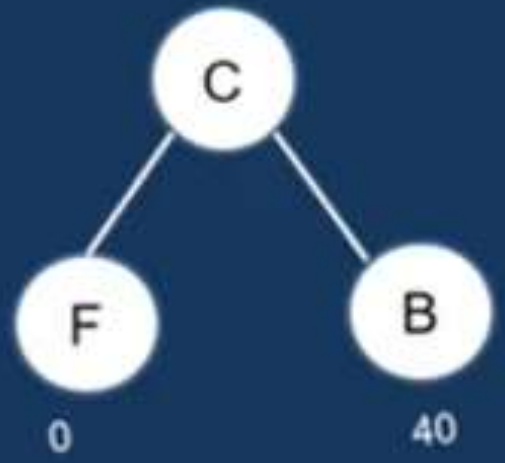
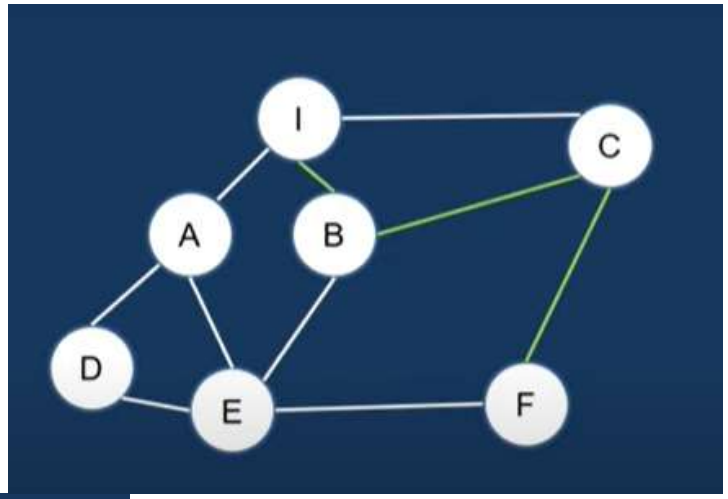


ILLUSTRATION OF BEST FIRST SEARCH





If we consider total distance travelled so far i.e. g(n) and heuristic value i.e. h(n), then there is high chance to get optimal solution.

A* ALGORITHM

- A* is the informed searching technique.
- It gives the guarantee of the optimal solution.
- We use the below equation in A* algorithm.

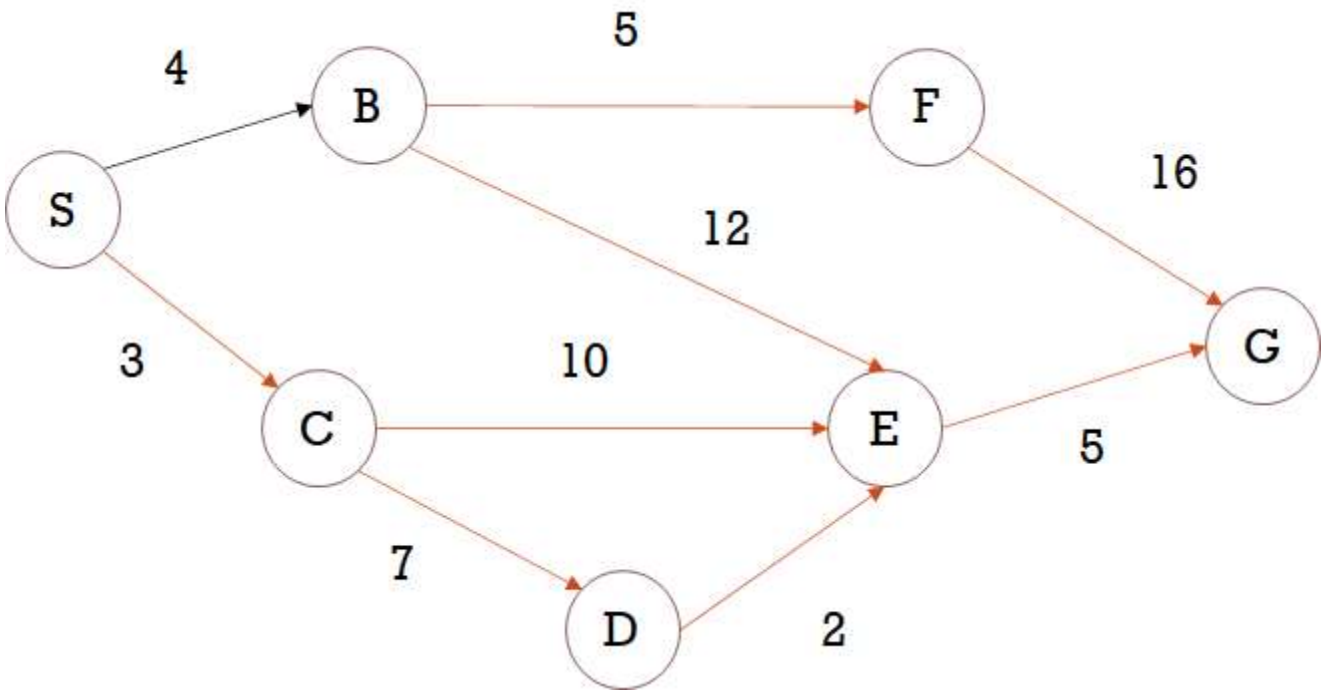
  $f(n)=g(n)+h(n)$

  g(n): the actual cost of traversal from initial state to the current state.

  h(n): the estimated cost of traversal from current state to goal state.

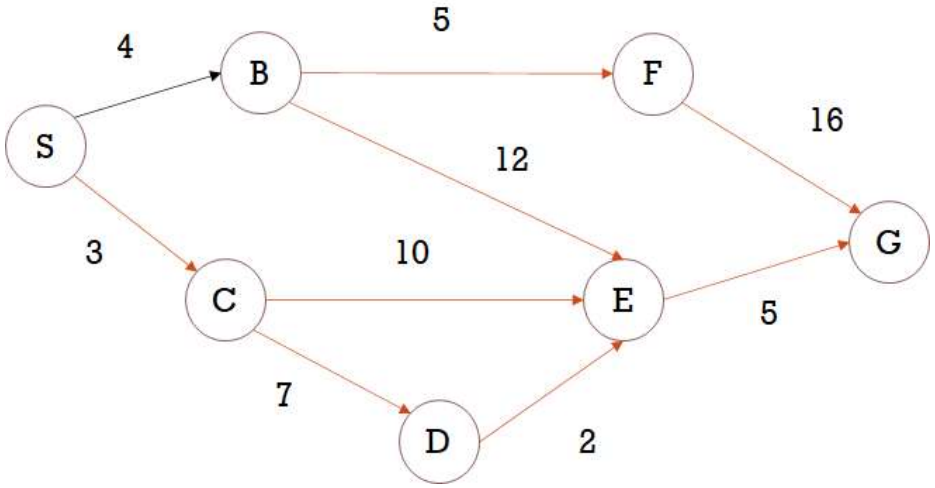
  f(n): the actual cost of traversal from initial state to goal state

ILLUSTRATION OF A*



S-G=14
B-G=12
F-G=11
C-G=11
E-G=4
D-G=6

ILLUSTRATION OF A*



S-G=14
B-G=12
F-G=11
C-G=11
E-G=4
D-G=6

f(s)= 0 + 14=14

S ⟶ B OR S ⟶ C

4+12= 16 3+11= 14

…….contd……>

SOLVING 8 PUZZLE WITH A* ALGORITHM

A* search uses two important values:

1. G: cost (no. of steps taken to current state from initial state)

2. H: estimated distance to goal (estimated no. of steps taken from current state to goal state)

To measure the H value use Manhattan Distance between two points/tiles.

Manhattan Distance= sum of vertical distance and horizonal distance

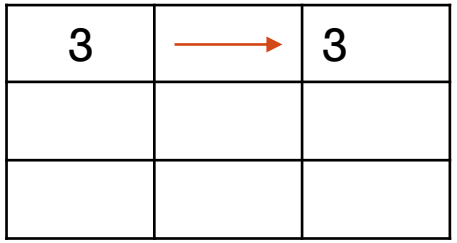
SOLVING 8 PUZZLE WITH A* ALGORITHM

3	2	8
4	5	6
7	1	

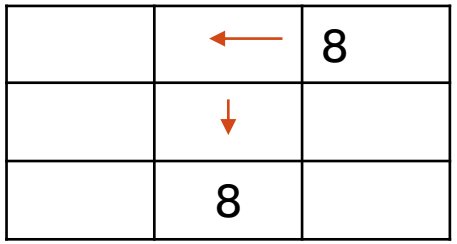
Current state

1	2	3
4	5	6
7	8	

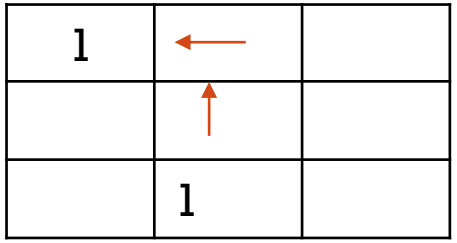
Goal state



Cost= 2 square



Cost= 3 square



Cost= 3 square

Total Cost (H)= 8

The H value is the sum of Manhattan Distance of all 8 squares except the empty square. Hence the H value for that state would be 8.

A* ALGORITHM

Real Life Applications in AI:

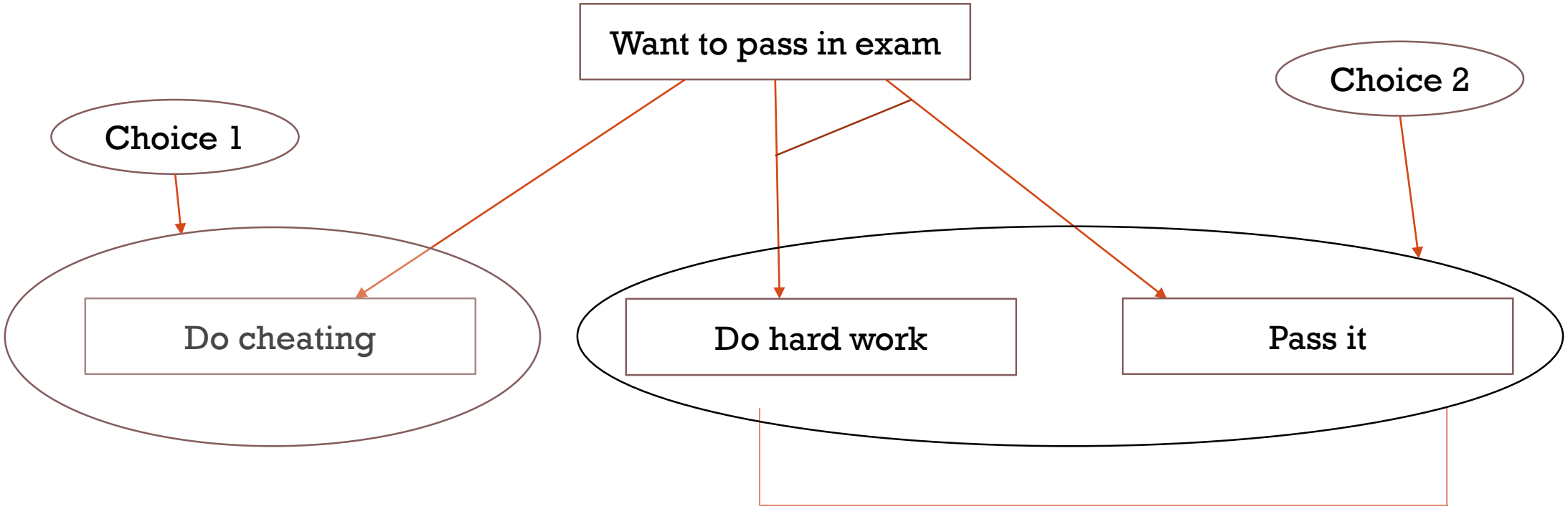
Search based problem where G and H values can be effectively calculated:

- Find the shortest path in Map
- Robotics- motion planning
- Solve puzzles

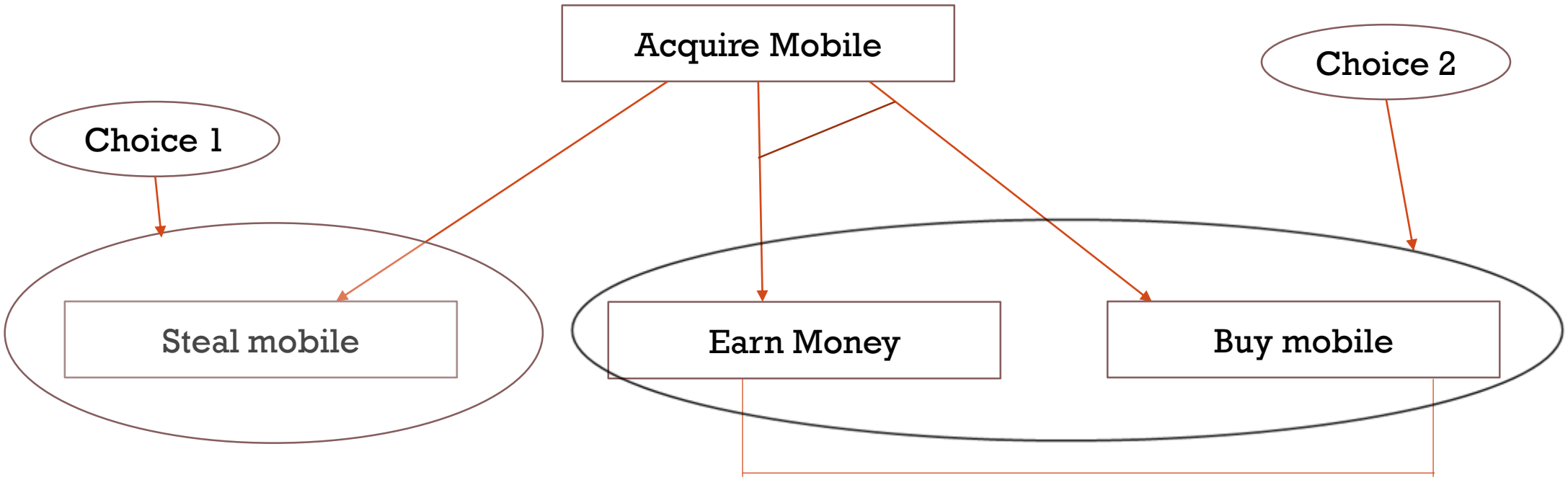
AO* ALGORITHM

- It is based on AND-OR graph
- It is the best first search algorithm
- It works on problem decomposition (breakdown into smaller pieces)- AO* uses the concept of AND-OR graphs to decompose any given problem into a set of smaller problems **which are further solved.**
- Examples:

AO* ALGORITHM: EXAMPLES



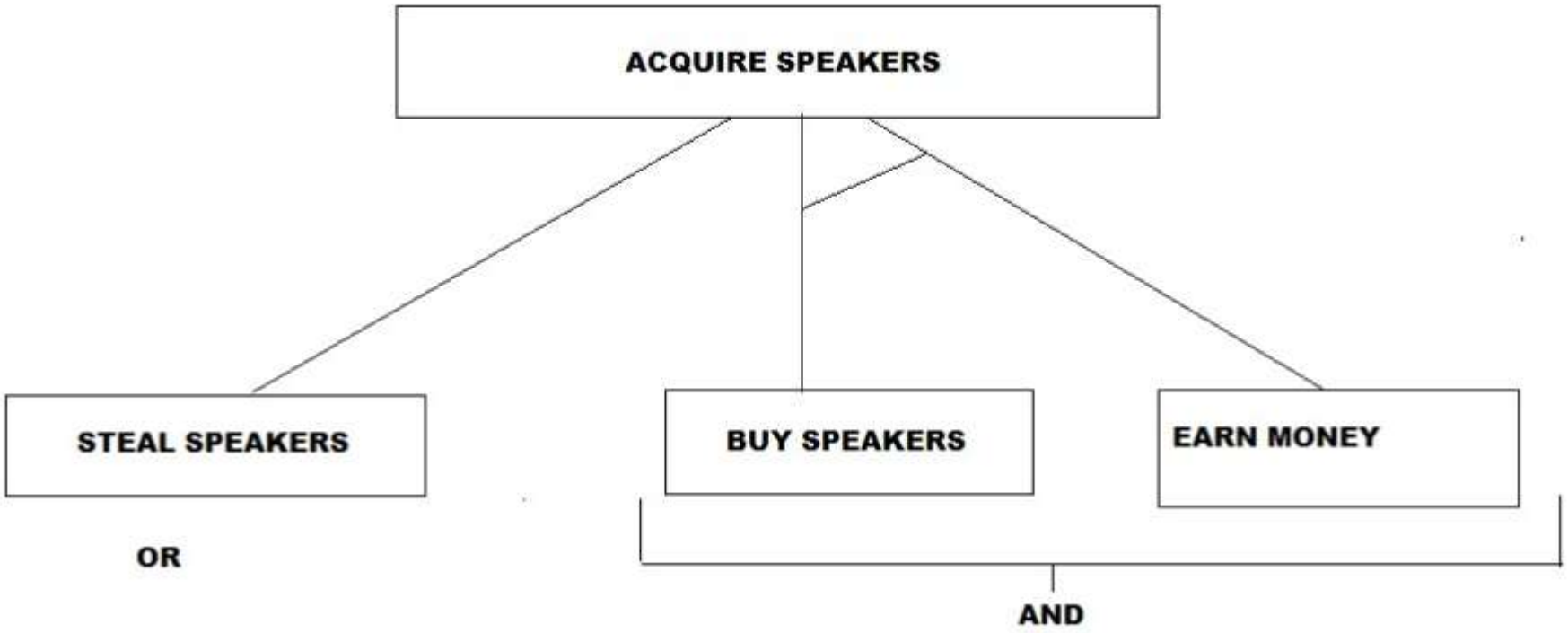
AO* ALGORITHM: EXAMPLES



OR

AND

AO* ALGORITHM: EXAMPLES

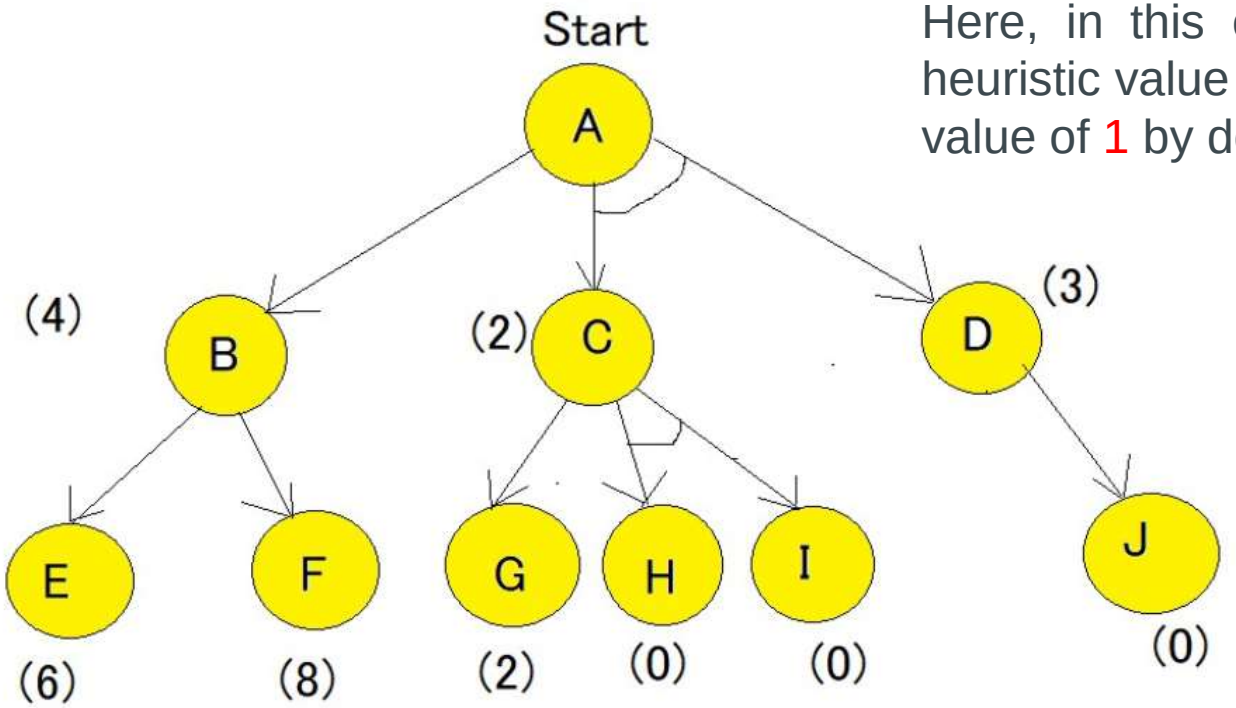


WORKING OF AO* ALGORITHM

f(n)=g(n)+h(n)

g(n): the actual cost of traversal from initial state to the current state.
h(n): the estimated cost of traversal from current state to goal state.
f(n): the actual cost of traversal from initial state to goal state

EXAMPLE-TO GET A BETTER IDEA OF THE AO* ALGORITHM



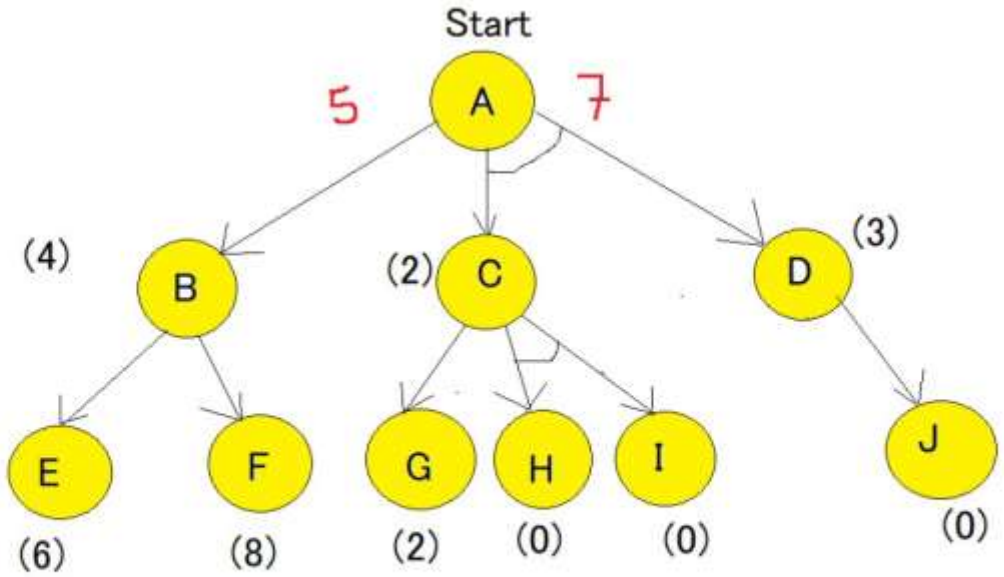
Here, in this example all numbers in brackets are the heuristic value i.e h(n). Each edge is considered to have a value of 1 by default.

Step-1

Starting from node A, we first calculate the best path.

f(A-B) = g(B) + h(B) = 1+4= 5 , where 1 is the default cost value of travelling from A to B and 4 is the estimated cost from B to Goal state.

f(A-C-D) = g(C) + h(C) + g(D) + h(D) = 1+2+1+3 = 7 , here we are calculating the path cost as both C and D because they have the AND-Arc. The default cost value of travelling from A-C is 1, and from A-D is 1, but the heuristic value given for C and D are 2 and 3 respectively hence making the cost as 7.



The minimum cost path is chosen i.e A-B.

Step-2

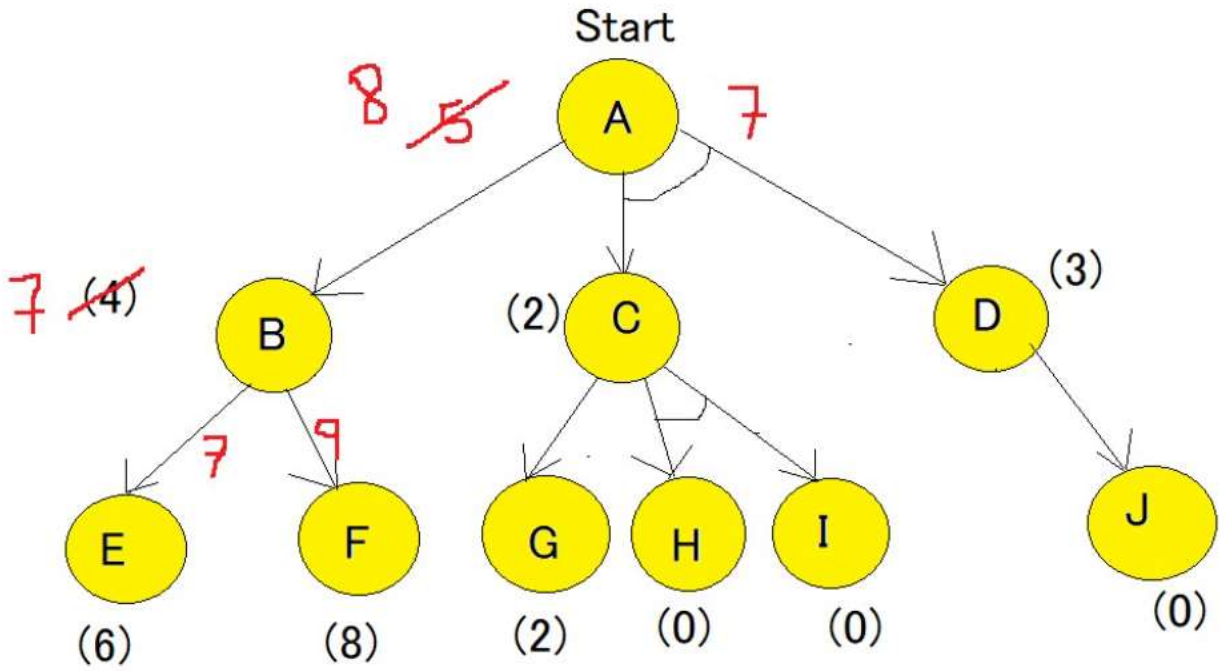
Using the same formula as step-1, the path is now calculated from the B node,

f(B-E) = 1 + 6 = 7.

f(B-F) = 1 + 8 = 9

Hence, the B-E path has lesser cost. Now the heuristics have to be updated since there is a difference between actual and heuristic value of B. The minimum cost path is chosen and is updated as the heuristic , in our case the value is 7. And because of change in heuristic of B there is also change in heuristic of A which is to be calculated again.

f(A-B) = g(B) + updated((h(B)) = 1+7=8



Step-3

Comparing path of f(A-B) and f(A-C-D) it is seen that f(A-C-D) is smaller. Hence f(A-C-D) needs to be explored.

Now the current node becomes C node and the cost of the path is calculated,

f(C-G) = 1+2 = 3

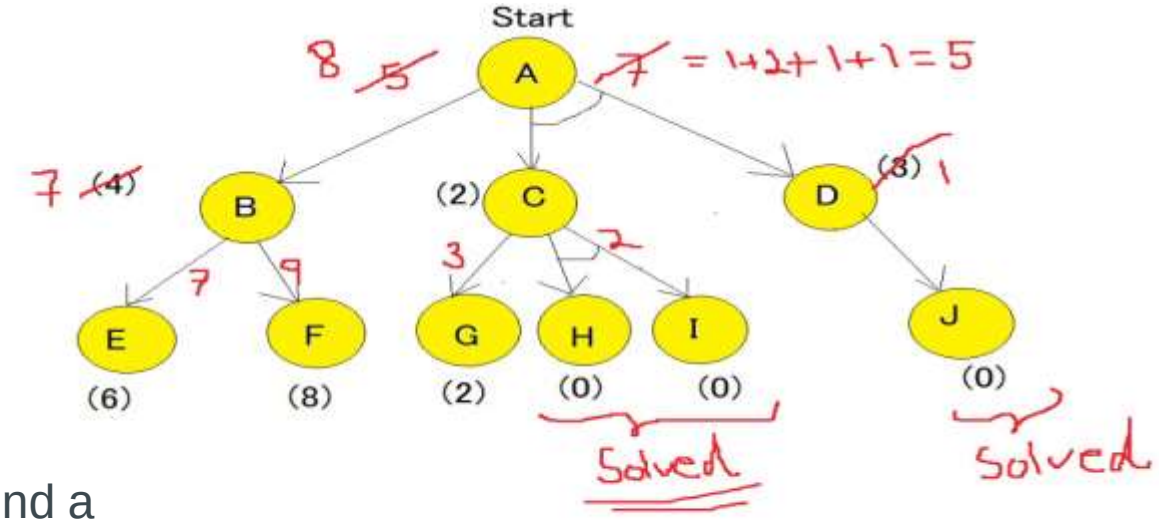
f(C-H-I) = 1+0+1+0 = 2

f(C-H-I) is chosen as minimum cost path,also there is no change in heuristic since it matches the actual cost.

Heuristic of path of H and I are 0 and hence they are solved, but Path A-D also needs to be calculated , since it has an AND-arc.

f(D-J) = 1+0 = 1, hence heuristic of D needs to be updated to 1. And finally the f(A-C-D) needs to be updated.

f(A-C-D) = g(C) + h(C) + g(D) + updated((h(D)) = 1+2+1+1 =5.



AO* algorithm will traverse through the problem and find a solution.

As we can see that the solved path is f(A-C-D).

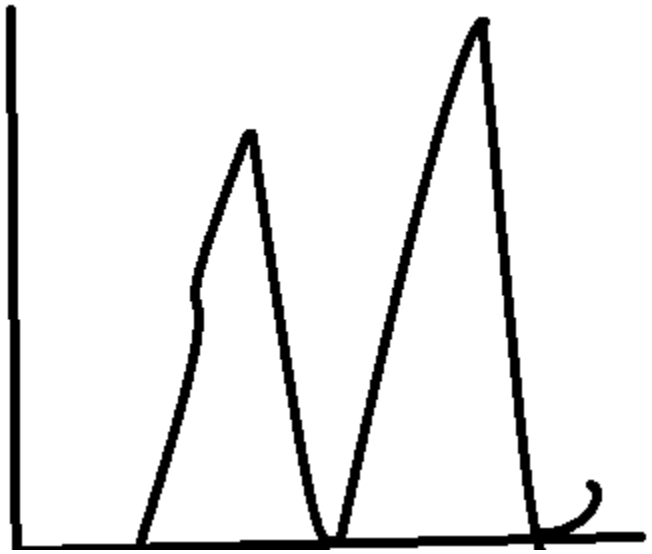
WHAT IS THE DIFFERENCE BETWEEN A* ALGORITHM AND AO* ALGORITHM?

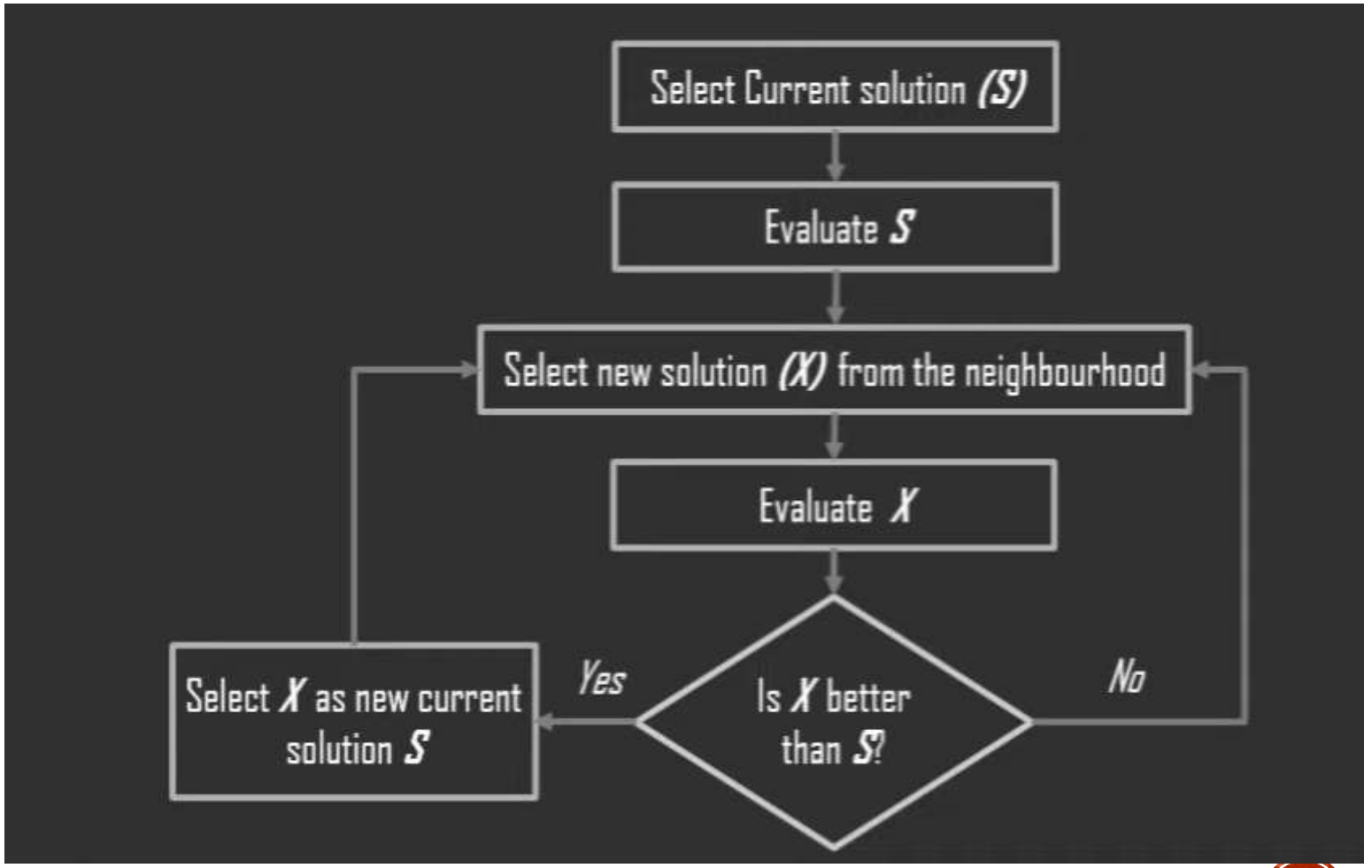
- A* algorithm provides with the optimal solution, whereas AO* stops when it finds any solution.
- AO* algorithm requires lesser memory compared to A* algorithm.
- AO* algorithm doesn't go into infinite loop whereas the A* algorithm can go into an infinite loop.

HILL CLIMBING

- Hill Climbing Algorithm (Local search algorithm: no knowledge of global domain, Greedy approach: whatever the best move for that it will work), no backtrack)
- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain OR best solution to the problem.
- It terminates when it reaches a peak value where no neighbour has a higher value.

FLOWCHART




Select Current solution (S)
Evaluate S
Select new solution (X) from the neighbourhood
Evaluate X
Is X better than S?
Yes
No
Select X as new current solution S

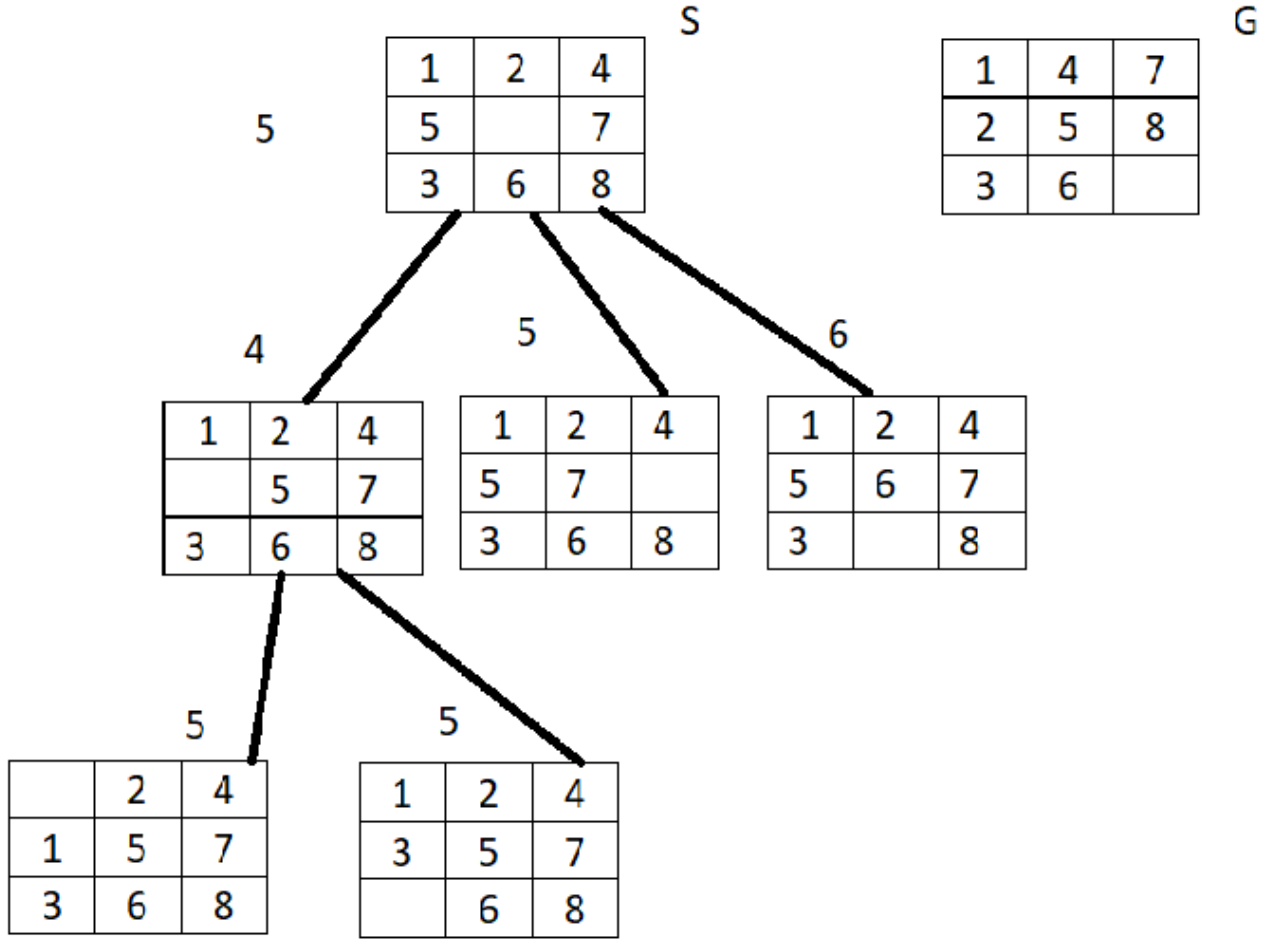
ALGORITHM

- Evaluate the initial state
- Loop until a solution is found OR there are no operators left
  - Select and apply a new operator
  - Evaluate the new state:

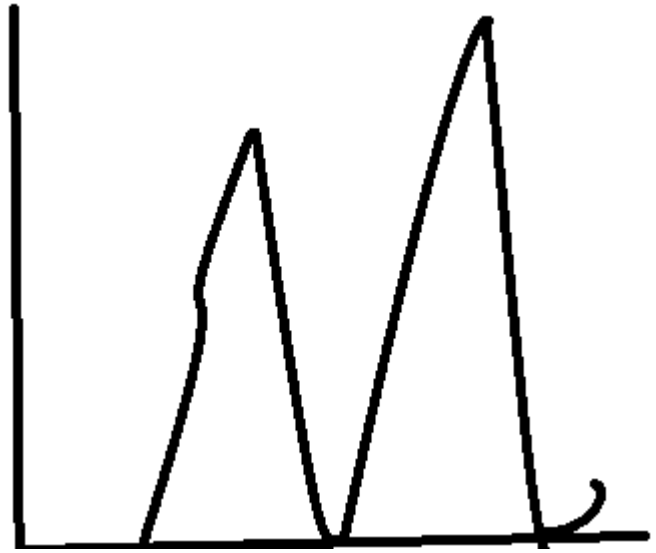
    if goal then quit

    if better than current state then it is new current state.

EXAMPLE

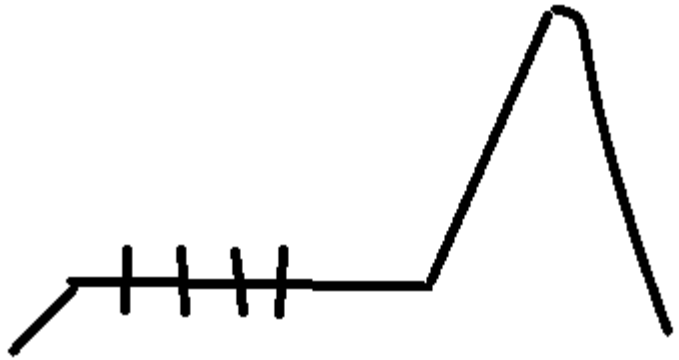


PROBLEMS IN HILL CLIMBING



Local maxima

PROBLEMS IN HILL CLIMBING

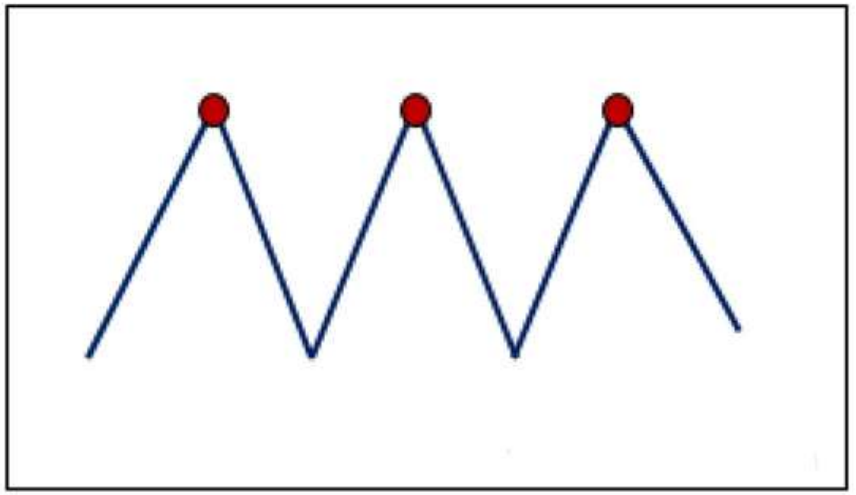


Plateau

PROBLEMS IN HILL CLIMBING

Ridge

- Ridge occurs when there are multiple peaks and all have the same value or in other words, there are multiple local maxima which are same as global maxima

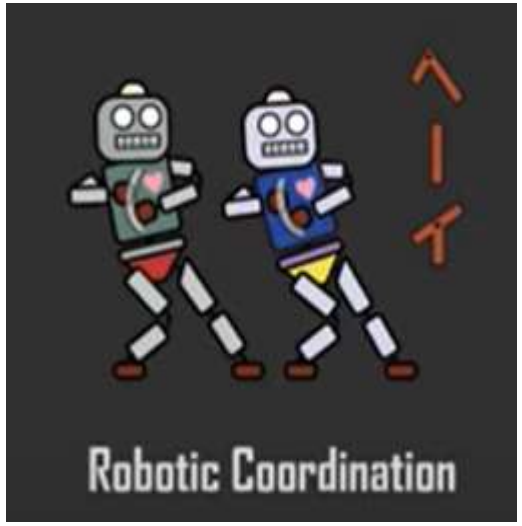


PROBLEMS IN HILL CLIMBING

- **Local maxima:** At a local maxima, the process will end even though a better solution may exist.
  - Solution: Utilize backtracking technique to deal with this situation.
- **Plateau:** On plateau all neighbors have same value,. Hence, it is not possible to select the best direction.
  - Solution: So make a big jump randomly select a state far away from the current state. Chances are that we will land at a non-plateau region.
- **Ridge:** A ridge can look like a peak; hence, algorithm can end.
  - Solution: In this kind of obstacle, use two or more rule before testing. It implies moving in several directions at once.

APPLICATIONS





Thanks